

COSC 4353 –Software Design

Group 1 – Assignment 3

GitHub Repository Link

- Directly to repository:
https://github.com/Antiquely3031/COSC4353_SoftwareDesignGroup1_Summer2026
- Directly to folder:
https://github.com/Antiquely3031/COSC4353_SoftwareDesignGroup1_Summer2026/tree/main/Assignment_3_Group_1

Group 1 – Team Contribution Table:

	Contributions
Patrick Callaghan	User/Admin Login Authentication framework and backend; unit test; setup destination directions for User/Admin dashboards; provided fake 'database' as placeholder; provided reasoning and design support responses in this report; edited/proofread submission document; test at 90% coverage
Elvis Noel Trujillo Chairez	Responsible for the Admin account backend and integration of said backend into the frontend; made the test suite for said backend; creator of this Word Document report (set up the outline of said document, and served as the main documentation lead of the group regarding this submission report). On the final note, for unit testing, the coverage was 100% across all metrics (statements, branches, functions, and lines).
Kevin Chau	User side of the queue management such as join/leaving; viewing current queue and serve next user; a mock database is implemented within the JavaScript file an API test with a sub 95% coverage was achieved
Richard Tiamzon	Notification module with backend trigger logic, and REST API, full input validation as well as API test (96% coverage); implemented an in-memory store as a placeholder database, the frontend toast/ list client, and linking the notifications between different modules.

Overview:

Backend language(s) and framework(s):

For the language(s), we had used JavaScript; whilst, for framework(s), we had used the following JavaScript frameworks/technologies: Express (Routing framework), Jest (general and universal unit testing framework), SuperTest (unit testing library for WebSocket), Fetch (API for HTTP capabilities), and WebSocket (API for TCP capabilities).

Reason(s) for the choice(s):

We chose to continue the JavaScript route because it matches what we completed with the frontend, and it allowed us to gain more experience building a backend-to-database connection on our own. We utilized the routing framework Express and opted out of using a service platform like Supabase or Firebase for any structural concerns. At this point in the project, we have decided to use MySQL as our database platform, but have included placeholders (e.g., fakeDB.js) to facilitate functionality prior to database creation. Regarding the login/authentication portion of the project, we chose bcrypt for password hashing, mostly because it is a standard password hashing element and never stores passwords. Jest and SuperTest are also pretty standard testing pieces for what we are completing. They allow us to describe, test, and expect what we need, as well as acquire coverage reports. We used REST-style routes not just because they were required in the instructions, but also because they provide a non-arbitrary method for error detection and status codes. Specifically, when wiring the forms to our backend, we used fetch() for API calls as that is native to the browser and avoids having to pull in an additional library. We chose WebSocket to handle the live updating requirements that are inherent in a queue management application.

How our choices support our design:

Exploring the Node/Express option instead of using a service platform like Supabase or Firebase allows us to generate validations on our own in addition to understanding the full scope of role handling. Our choice to eventually use MySQL supports the actual shape (relational model) of the data we intend to process. The placeholders give us the option to build our entire API surface without being tied down to database concerns. We need a legitimate way to validate basic password input and bcrypt is superior to any plain-text password checking. Jest and SuperTest satisfy the unit testing element of the project, giving us something that evaluates the “business logic” and “coverage.” The one-shot authentication requests that are part of the login/authentication portion of the project are more easily addressed with fetch() than other processes, while WebSocket better supports the concurrency and updating issues that will assuredly need addressing. Essentially, we believe our choices make each piece of our budding application independently correct, testable, and provable.